Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

AI Vibe Coding for Multi-Agents System

TGS Ref No: TGS-2020503207

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.1

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	11 August 2026	Initial release — Learner Guide for AI Vibe Coding for Multi-Agents System covering all 11 hands-on labs across the five topics of the published course outline.	Dr Alfred Ang
1.1	11 August 2026	Added a 'Where the code lives' section to every lab with its folder path and Google Colab link, following the restructure of the labs into one folder per lab, each holding a runnable Python script and a Colab notebook.	Dr Alfred Ang

TABLE OF CONTENTS

Introduction

This Learner Guide accompanies the WSQ course AI Vibe Coding for Multi-Agents System (TGS-2020503207), conducted by Tertiary Infotech Academy Pte Ltd. It provides the detailed step-by-step instructions for all 11 hands-on labs, organised by the five topics of the published course outline. Every lab maps to a learning outcome and is completed in Python on your own laptop.

Use this guide alongside the course slides. The slide deck is deliberately visual — it shows the architecture, the workflow and what each lab produces — while this guide is the single place where the full step-by-step instructions and commands are recorded. Work through the steps here as the trainer walks through the concepts on the slides.

All lab code is available in the course repository at https://github.com/tertiarycourses/TGS-2020503207-AI-Vibe-Coding-for-Multi-Agents-System. Keep this guide open during the assessment: the final assessment is open book.

Course Learning Outcomes

- LO1: Explain the components of a modern AI agent — skills, memory, tools and the Model Context Protocol (MCP) — and the progression from single-agent to multi-agent collaboration.
- LO2: Apply a structured vibe coding workflow with context engineering to specify, generate and verify agent code reliably.
- LO3: Build a collaborative multi-agent system with the OpenAI Agents SDK using structured outputs, tool calling and supervisor routing, and deploy it with Streamlit.
- LO4: Build a collaborative multi-agent system with the Google Gemini Agent SDK and deploy it with Streamlit.
- LO5: Orchestrate tools through MCP and design hierarchical sub-agent architectures for delegated, specialised work.

Skills Framework Alignment

TSC Title: Analytics and Computational Modelling | TSC Code: ICT-DIT-3001-1.1

- A1 - Identify appropriate statistical algorithms and data models to test hypotheses or theories
- A2 - Use appropriate analytics platforms and analytical tools given specific analytics and reporting requirements
- A3 - Utilise a range of statistical methods and analytics approaches to analyse data
- A4 - Evaluate the performance and suitability of models against business requirements

Before You Start — Environment Setup

What you need

- A laptop (Windows, macOS or Linux) with Python 3.11 or later installed.
- A terminal and a code editor — VS Code is recommended.
- An OpenAI API key from platform.openai.com (used in Topics 1, 3 and 5).
- A Google AI Studio API key from aistudio.google.com (used in Topic 4).
- An AI coding agent for the vibe coding labs — Claude Code, Gemini CLI, Codex CLI, Cursor or Windsurf.
- Git, for version control during the vibe coding labs.
- A stable internet connection — every lab calls a hosted model API.

Create your working environment

Create one project folder for the whole course with a single virtual environment. Every lab builds on the same environment, so you install the dependencies once.

```
# create the course folder and a virtual environment

mkdir multi-agent-course && cd multi-agent-course

python3 -m venv .venv

source .venv/bin/activate        # Windows: .venv\Scripts\activate


# install everything the labs need

pip install openai openai-agents google-adk pydantic python-dotenv
streamlit "mcp[cli]"
```

Store your API keys safely

Never hard-code an API key in source code, and never commit one to Git. Put your keys in a .env file and load them with python-dotenv. Add .env to .gitignore before your first commit.

```
# .env  (never commit this file)

OPENAI_API_KEY=sk-...

GOOGLE_API_KEY=...


# .gitignore

.env

.venv/
```

```
__pycache__/

memory.json
```

Verify the setup

Confirm Python and the key packages are importable before the first lab. If any import fails, re-activate the virtual environment and re-run the install command.

```
python3 --version

python3 -c "import openai, agents, pydantic, streamlit; print('OK')"

python3 -c "from google.adk.agents import Agent; print('ADK OK')"

python3 -c "from mcp.server.fastmcp import FastMCP; print('MCP OK')"
```

How the labs are organised

Every lab has its own folder under labs/ in the course repository, named lab-01-…, lab-02-… and so on. Each folder contains a runnable Python script, a README with the lab instructions, and a Jupyter notebook you can open directly in Google Colab if you would rather not install anything locally.

- Run locally: activate the virtual environment, ensure your .env holds the API keys, then run the script named in that lab's README (for example: python agent.py).
- Run in Colab: open the lab's README on GitHub and click the Open in Colab badge. Put your API keys in Colab Secrets rather than typing them into a cell.
- The two vibe coding labs (Labs 3 and 4) have no script to run — you direct an AI coding agent to write the code, so their folders hold the specification and context files instead.

Conventions used in every lab

- Commands are run from your terminal with the virtual environment activated.
- Placeholders such as <YOUR_KEY> and /absolute/path/ are replaced with your own values.
- Model names change over time — if a model identifier is rejected, check the provider's current model list.
- Cost control: every lab uses small prompts, but keep an eye on your provider usage dashboard.

Topic 01 — Modern Agent Foundations

Agent anatomy · Skills · Memory · MCP · Single-agent to multi-agent

Key concepts

- What is a modern AI agent? — An LLM that reasons in a loop, calls tools, keeps memory and pursues a goal — not a single prompt-and-response.
- Agent skills — Modular, transferable capabilities packaged so an agent can load only what a task needs, and reuse them across projects.
- Agent memory — Short-term context versus long-term stores; what to remember, what to summarise and what to discard.
- Model Context Protocol (MCP) — An open standard that lets an agent discover and call external tools and data sources through a uniform interface.
- Tools and tool calling — The agent chooses a function, the runtime executes it, and the result re-enters the reasoning loop.
- Single-agent to multi-agent — When one agent's context and responsibilities grow too large, split the work across specialised collaborating agents.

Lab 1 — Build Your First Tool-Calling Agent

Learning outcome: Explain the components of a modern AI agent and implement a reasoning loop with tool calling.

Goal: Build a single agent from first principles so the loop is not a black box: the model receives a goal, decides to call a tool, your code executes it, and the result is fed back until the agent can answer. You will see every message that crosses the boundary.

What you'll build

A runnable Python agent that answers questions by calling two of your own tools, printing each reasoning step and tool result so the loop is visible. (Tools: Python 3.11+, OpenAI Python SDK, uv/pip, .env for API keys.)

Where the code lives

- Lab folder: labs/lab-01-build-your-first-tool-calling-agent/ — the runnable Python script, a README and the notebook.
- Run it locally: activate your virtual environment, set your keys in .env, then run the script named in the lab's README.
- Or open it in Google Colab: https://colab.research.google.com/github/tertiarycourses/TGS-2020503207-AI-Vibe-Coding-for-Multi-Agents-System/blob/main/labs/lab-01-build-your-first-tool-calling-agent/lab-01-build-your-first-tool-calling-agent.ipynb

Step-by-step

1. Create the project folder and a virtual environment, then install the OpenAI SDK and python-dotenv.

```
uv venv && source .venv/bin/activate && uv pip install openai python-dotenv
```

2. Store your API key in a .env file (never hard-code a key in source, and never commit .env).

```
echo 'OPENAI_API_KEY=sk-...' > .env
```

3. Write two plain Python functions the agent may call — get_weather(city) and calculate(expression) — each returning a JSON-serialisable result.
4. Describe both functions as JSON tool schemas: name, description and typed parameters. The description is what the model uses to decide when to call it.
5. Write the agent loop: send messages plus tools to the model, and if the reply contains tool_calls, execute the matching function and append the result as a tool message.
6. Cap the loop with a maximum number of turns so a confused agent cannot spin forever.
7. Run the agent and print each step to observe the reason-act-observe cycle end to end.

```
python agent.py "What is the weather in Singapore, and what is 15% of
2400?"
```

Test it

The agent answers both parts of the question in one run, and the printed trace shows it called get_weather once and calculate once before producing the final answer.

Note: The complete working code for this lab is in labs/lab-01-build-your-first-tool-calling-agent/ in the course repository. Keep your API keys in .env — never commit them.

Lab 2 — Agent Memory and Reusable Agent Skills

Learning outcome: Implement short-term and long-term agent memory, and package a capability as a reusable skill.

Goal: Give the agent memory so it stops forgetting between turns, then extract a repeatable procedure into a skill file the agent loads on demand — the modular, transferable capability described in the course outline.

What you'll build

An agent with a conversation buffer plus a persistent JSON memory store, and one reusable skill file that the agent loads only when the task calls for it. (Tools: Python, OpenAI SDK, JSON file store, Markdown skill definition.)

Where the code lives

- Lab folder: labs/lab-02-agent-memory-and-reusable-agent-skills/ — the runnable Python script, a README and the notebook.
- Run it locally: activate your virtual environment, set your keys in .env, then run the script named in the lab's README.

- Or open it in Google Colab: https://colab.research.google.com/github/tertiarycourses/TGS-2020503207-AI-Vibe-Coding-for-Multi-Agents-System/blob/main/labs/lab-02-agent-memory-and-reusable-agent-skills/lab-02-agent-memory-and-reusable-agent-skills.ipynb

Step-by-step

1. Add a messages list as short-term memory so the agent can resolve follow-up questions such as 'and what about tomorrow?'.
2. Add a long-term store: remember_fact(key, value) and recall_facts() writing to memory.json, both exposed to the agent as tools.
3. Inject the recalled facts into the system prompt at the start of each run so memory actually influences behaviour.
4. Summarise older turns once the buffer grows past a threshold, keeping the context small and cheap.
5. Write a skill as a Markdown file with a name, a description of when to use it, and the procedure steps the agent should follow.
6. Load the skill into the system prompt only when the user's request matches its description, and observe the difference in output quality.

Test it

Tell the agent a fact, restart the process, ask about it again — the agent recalls it from memory.json. With the skill loaded, the agent follows the documented procedure instead of improvising.

Note: The complete working code for this lab is in labs/lab-02-agent-memory-and-reusable-agent-skills/ in the course repository. Keep your API keys in .env — never commit them.

Topic 02 — Vibe Coding for Multi-Agent Systems

Vibe coding workflow · Context engineering · Tooling · Agent skills

Key concepts

- What is vibe coding? — Directing an AI coding agent in natural language to generate, run and refine working software, while you stay the reviewer and architect.
- The structured workflow — Specify, then scaffold, then generate, then run, then verify, then refine — a disciplined loop, not one-shot prompting.
- Context engineering — Curating exactly the files, specs and constraints the agent sees, which drives reliability far more than prompt wording.
- Vibe coding tools — Claude Code, Gemini CLI, Codex CLI, Cursor, Windsurf, Antigravity and Trae — terminal agents and agentic IDEs.

- Agent skills integration — Packaging repeatable procedures as skills the coding agent invokes, so quality does not depend on re-explaining each time.
- Human in the loop — You own the specification, the review and the acceptance test; the agent owns the typing.

Lab 3 — Vibe Code an Agent with a Structured Workflow

Learning outcome: Apply the structured vibe coding workflow — specify, scaffold, generate, run, verify, refine.

Goal: Use an AI coding agent to build a working application without hand-writing the implementation, following a disciplined loop rather than one-shot prompting. The specification you write is the real deliverable; the agent produces the code.

What you'll build

A working research-assistant agent generated by a coding agent from your written specification, with an acceptance test you defined before any code existed. (Tools: Claude Code / Gemini CLI / Cursor, Python, Git.)

Where the code lives

- Lab folder: labs/lab-03-vibe-code-an-agent-with-a-structured-workflow/ — the runnable Python script, a README and the notebook.
- Run it locally: activate your virtual environment, set your keys in .env, then run the script named in the lab's README.
- Or open it in Google Colab: https://colab.research.google.com/github/tertiarycourses/TGS-2020503207-AI-Vibe-Coding-for-Multi-Agents-System/blob/main/labs/lab-03-vibe-code-an-agent-with-a-structured-workflow/lab-03-vibe-code-an-agent-with-a-structured-workflow.ipynb

Step-by-step

1. Install and launch your vibe coding tool in an empty project folder, then initialise Git so every AI change is reviewable as a diff.

```
git init && claude
```

2. Write SPEC.md first: the goal, the inputs and outputs, the constraints, and the acceptance test. Be explicit about what 'done' means.
3. Ask the agent to scaffold only the project structure and dependencies from SPEC.md — structure first, logic second.
4. Direct the agent to implement one component at a time, reviewing the diff after each before moving on.
5. Run the code and paste real error output back to the agent — concrete errors produce far better fixes than 'it does not work'.

6. Verify against the acceptance test in SPEC.md, then commit the working state before the next iteration.

```
git add -A && git commit -m 'Working research agent per SPEC.md'
```

7. Refine one improvement at a time, re-running the acceptance test after each change.

Test it

The generated agent passes the acceptance test you wrote in SPEC.md before any code existed, and the Git history shows small reviewable commits rather than one giant unreviewed dump.

Note: The complete working code for this lab is in labs/lab-03-vibe-code-an-agent-with-a-structured-workflow/ in the course repository. Keep your API keys in .env — never commit them.

Lab 4 — Context Engineering and a Reusable Coding Skill

Learning outcome: Engineer the agent's context and package a repeatable procedure as a project skill.

Goal: Improve reliability by controlling exactly what the coding agent sees, then capture your house conventions as a skill so the agent applies them automatically instead of you re-explaining them in every session.

What you'll build

A project configured with a context file and a reusable skill, demonstrably producing house-standard code without repeated instructions. (Tools: Claude Code / Gemini CLI, Markdown, Git.)

Where the code lives

- Lab folder: labs/lab-04-context-engineering-and-a-reusable-coding-skill/ — the runnable Python script, a README and the notebook.
- Run it locally: activate your virtual environment, set your keys in .env, then run the script named in the lab's README.
- Or open it in Google Colab: https://colab.research.google.com/github/tertiarycourses/TGS-2020503207-AI-Vibe-Coding-for-Multi-Agents-System/blob/main/labs/lab-04-context-engineering-and-a-reusable-coding-skill/lab-04-context-engineering-and-a-reusable-coding-skill.ipynb

Step-by-step

1. Take a deliberately vague prompt and note the weaknesses in what the agent produces — this is your baseline.
2. Create a project context file (CLAUDE.md or GEMINI.md) stating the stack, conventions, folder layout and what the agent must never do.

3. Re-run the same vague prompt and compare the output against the baseline to see context alone change the result.
4. Write a skill file capturing a repeatable procedure — for example 'add a new agent tool' — with its steps and quality checks.
5. Reference precise files and line ranges in your next request instead of pasting whole files, keeping the context small and relevant.
6. Ask the agent to perform the procedure and confirm it follows the skill rather than improvising.

Test it

With the context file and skill in place, the same prompt that previously produced inconsistent code now yields code matching your stated conventions, without you restating them.

Note: The complete working code for this lab is in labs/lab-04-context-engineering-and-a-reusable-coding-skill/ in the course repository. Keep your API keys in .env — never commit them.

Topic 03 — Multi-Agent System Development with OpenAI Agents SDK

Agent design · Structured outputs · Tool calling · Supervisor routing · Streamlit

Key concepts

- The OpenAI Agents SDK — A lightweight Python framework for agents, tools, handoffs and guardrails, with tracing built in.
- Agents and instructions — An agent is a model plus instructions plus a tool set; clear role boundaries make multi-agent behaviour predictable.
- Structured outputs — Pydantic output types force the model to return validated, typed data your code can rely on.
- Function tools and API calls — Decorate a Python function as a tool so the agent can call your own logic and external APIs.
- Supervisor routing and handoffs — A triage agent classifies the request and delegates to the specialist sub-agent best suited to it.
- Streamlit deployment — Wrap the multi-agent system in a simple web UI so non-developers can use it.

Lab 5 — First Agent with the OpenAI Agents SDK

Learning outcome: Design an agent with the OpenAI Agents SDK using structured outputs and function tools.

Goal: Rebuild the hand-rolled agent from Lab 1 on the OpenAI Agents SDK and see how much the framework removes. Add a Pydantic output type so the agent returns validated typed data instead of prose, and expose your own Python functions as tools.

What you'll build

An SDK-based agent that calls your function tools and returns a validated Pydantic object your code can use directly. (Tools: Python 3.11+, openai-agents SDK, Pydantic, .env.)

Where the code lives

- Lab folder: labs/lab-05-first-agent-with-the-openai-agents-sdk/ — the runnable Python script, a README and the notebook.
- Run it locally: activate your virtual environment, set your keys in .env, then run the script named in the lab's README.
- Or open it in Google Colab: https://colab.research.google.com/github/tertiarycourses/TGS-2020503207-AI-Vibe-Coding-for-Multi-Agents-System/blob/main/labs/lab-05-first-agent-with-the-openai-agents-sdk/lab-05-first-agent-with-the-openai-agents-sdk.ipynb

Step-by-step

1. Install the Agents SDK and Pydantic into your virtual environment.

```
uv pip install openai-agents pydantic python-dotenv
```

2. Create an Agent with a name, clear instructions and a model, then run it with Runner.run_sync to confirm the setup works.
3. Define a Pydantic BaseModel for the answer shape you want, and set it as the agent's output_type.
4. Decorate a Python function with @function_tool — the docstring and type hints become the schema the model sees.
5. Add a second tool that calls a real public API so the agent works with live data rather than hard-coded values.
6. Inspect the run result: the final validated output, plus which tools were called and in what order.
7. Handle the failure path — a tool that raises should not crash the agent run.

Test it

result.final_output is an instance of your Pydantic model with correctly typed fields, and the run trace shows the expected tool calls.

Note: The complete working code for this lab is in labs/lab-05-first-agent-with-the-openai-agents-sdk/ in the course repository. Keep your API keys in .env — never commit them.

Lab 6 — Supervisor Routing and Sub-Agent Delegation

Learning outcome: Construct a collaborative multi-agent system with supervisor routing and handoffs.

Goal: Move from one overloaded agent to a team. Build three specialists and a triage supervisor that classifies each request and hands it to the right one — the core multi-agent pattern of the course.

What you'll build

A four-agent system — a triage supervisor plus research, coding and writing specialists — that routes each request to the correct specialist. (Tools: Python, openai-agents SDK, Pydantic.)

Where the code lives

- Lab folder: labs/lab-06-supervisor-routing-and-sub-agent-delegation/ — the runnable Python script, a README and the notebook.
- Run it locally: activate your virtual environment, set your keys in .env, then run the script named in the lab's README.
- Or open it in Google Colab: https://colab.research.google.com/github/tertiarycourses/TGS-2020503207-AI-Vibe-Coding-for-Multi-Agents-System/blob/main/labs/lab-06-supervisor-routing-and-sub-agent-delegation/lab-06-supervisor-routing-and-sub-agent-delegation.ipynb

Step-by-step

1. Define three specialist agents, each with narrow instructions and only the tools its own job requires.
2. Define the triage agent and pass the specialists in its handoffs list.
3. Write triage instructions describing when to route to each specialist, with a default for ambiguous requests.
4. Run three different requests and confirm each reaches the intended specialist.
5. Add a guardrail that rejects out-of-scope requests before any specialist work begins.
6. Inspect the trace to see the handoff chain and where the time and tokens were spent.
7. Compare against a single agent given all the tools, and note the difference in reliability.

Test it

A research question reaches the research agent, a coding request reaches the coding agent, and an out-of-scope request is refused by the guardrail before any specialist runs.

Note: The complete working code for this lab is in labs/lab-06-supervisor-routing-and-sub-agent-delegation/ in the course repository. Keep your API keys in .env — never commit them.

Lab 7 — Deploy the Multi-Agent System with Streamlit

Learning outcome: Deploy a collaborative multi-agent system as a shareable Streamlit web application.

Goal: Put a web interface on the agent team from Lab 6 so a non-developer can use it, streaming the reply and showing which agent handled each request.

What you'll build

A running Streamlit chat application backed by your multi-agent system, with visible routing and persistent conversation history. (Tools: Python, Streamlit, openai-agents SDK.)

Where the code lives

- Lab folder: labs/lab-07-deploy-the-multi-agent-system-with-streamlit/ — the runnable Python script, a README and the notebook.
- Run it locally: activate your virtual environment, set your keys in .env, then run the script named in the lab's README.
- Or open it in Google Colab: https://colab.research.google.com/github/tertiarycourses/TGS-2020503207-AI-Vibe-Coding-for-Multi-Agents-System/blob/main/labs/lab-07-deploy-the-multi-agent-system-with-streamlit/lab-07-deploy-the-multi-agent-system-with-streamlit.ipynb

Step-by-step

1. Install Streamlit and create app.py alongside your agents module.

```
uv pip install streamlit && streamlit run app.py
```

2. Build the chat UI with st.chat_message and st.chat_input.
3. Keep the conversation in st.session_state so history survives Streamlit's re-run on every interaction.
4. Call the triage agent from the UI and render the final output.
5. Display which specialist handled the request, so the routing is visible to the user.
6. Stream the response so the user sees progress instead of a frozen page.
7. Read the API key from the environment, never from the source, and confirm .env is git-ignored.

Test it

The app runs at localhost:8501, answers a multi-turn conversation with history intact, names the specialist that handled each turn, and contains no hard-coded API key.

Note: The complete working code for this lab is in labs/lab-07-deploy-the-multi-agent-system-with-streamlit/ in the course repository. Keep your API keys in .env — never commit them.

Topic 04 — Multi-Agent System Development with Gemini Agent SDK

Gemini agent design · Collaborative agents · Streamlit deployment

Key concepts

- The Google Gemini Agent SDK — Google's Agent Development Kit (ADK) for building, composing and running Gemini-powered agents in Python.
- Defining a Gemini agent — Model, instruction, description and tools — the description is what makes an agent discoverable by its coordinator.
- Function tools in ADK — Plain Python functions become tools; the docstring and type hints tell the model how and when to call them.
- Multi-agent composition — Sub-agents attached to a coordinator agent, which routes each request to the right specialist.
- Sessions and runners — The runner executes the agent loop while the session service carries conversational state.
- Comparing the SDKs — The same multi-agent pattern expressed in two ecosystems — portable concepts, different APIs.

Lab 8 — Build a Multi-Agent System with the Gemini Agent SDK

Learning outcome: Design collaborative agents with the Google Gemini Agent SDK (ADK).

Goal: Express the same multi-agent pattern in Google's ecosystem. Building the equivalent system twice separates the portable concepts — agents, tools, routing — from one vendor's API surface.

What you'll build

A Gemini-powered coordinator agent with two specialist sub-agents and working function tools. (Tools: Python 3.11+, google-adk, Google AI Studio API key.)

Where the code lives

- Lab folder: labs/lab-08-build-a-multi-agent-system-with-the-gemini-agent-sdk/ — the runnable Python script, a README and the notebook.
- Run it locally: activate your virtual environment, set your keys in .env, then run the script named in the lab's README.
- Or open it in Google Colab: https://colab.research.google.com/github/tertiarycourses/TGS-2020503207-AI-Vibe-Coding-for-Multi-Agents-System/blob/main/labs/lab-08-build-a-multi-agent-system-

with-the-gemini-agent-sdk/lab-08-build-a-multi-agent-system-with-the-gemini-agent-sdk.ipynb

Step-by-step

1. Install the Google Agent Development Kit and set your Gemini API key in .env.

```
uv pip install google-adk
```

2. Create an Agent with name, model, description and instruction — the description is what makes it discoverable by a coordinator.
3. Write plain Python functions as tools; ADK reads the type hints and docstring to build the schema.
4. Run the agent with a Runner and an in-memory session service to confirm single-agent behaviour.
5. Create two specialist sub-agents with distinct descriptions and non-overlapping tools.
6. Attach them to a coordinator agent via sub_agents and let it route by description.
7. Run requests that should reach different specialists and confirm the routing.
8. Note the API differences from the OpenAI SDK while the architecture stays identical.

Test it

The coordinator routes each request to the correct sub-agent, and the tools return live results — the same behaviour as the OpenAI build, in a different SDK.

Note: The complete working code for this lab is in labs/lab-08-build-a-multi-agent-system-with-the-gemini-agent-sdk/ in the course repository. Keep your API keys in .env — never commit them.

Lab 9 — Deploy the Gemini Multi-Agent System with Streamlit

Learning outcome: Deploy a Gemini-based collaborative multi-agent system with Streamlit.

Goal: Give the Gemini agent team the same web interface treatment, then compare the two deployments side by side and record which ecosystem fits which kind of work.

What you'll build

A deployed Streamlit application backed by the Gemini multi-agent system, plus a short written comparison of the two SDKs. (Tools: Python, Streamlit, google-adk.)

Where the code lives

- Lab folder: labs/lab-09-deploy-the-gemini-multi-agent-system-with-streamlit/ — the runnable Python script, a README and the notebook.

- Run it locally: activate your virtual environment, set your keys in .env, then run the script named in the lab's README.
- Or open it in Google Colab: https://colab.research.google.com/github/tertiarycourses/TGS-2020503207-AI-Vibe-Coding-for-Multi-Agents-System/blob/main/labs/lab-09-deploy-the-gemini-multi-agent-system-with-streamlit/lab-09-deploy-the-gemini-multi-agent-system-with-streamlit.ipynb

Step-by-step

1. Create a Streamlit app that imports your Gemini coordinator agent.
2. Manage the ADK session and runner inside st.session_state so state survives re-runs.
3. Render the chat history and stream the coordinator's reply.
4. Show which sub-agent handled each turn.
5. Add a sidebar control to switch Gemini model variants and observe the latency and quality trade-off.
6. Run both the OpenAI and Gemini apps and compare routing quality, latency and developer experience.
7. Record your comparison in README.md as evidence for the practical assessment.

Test it

The Gemini app runs, routes correctly and holds conversation state, and your README records a concrete comparison of the two SDKs.

Note: The complete working code for this lab is in labs/lab-09-deploy-the-gemini-multi-agent-system-with-streamlit/ in the course repository. Keep your API keys in .env — never commit them.

Topic 05 — MCP and Sub-Agents

MCP servers · Tool orchestration · Hierarchical sub-agent architecture

Key concepts

- MCP in depth — Servers expose tools, resources and prompts; clients (your agent, Claude Code) discover and invoke them over a standard protocol.
- Why MCP matters — Write a capability once as a server, and every MCP-aware agent can use it — no bespoke integration per framework.
- Building an MCP server — Define typed tools with FastMCP and run them over stdio for local agents.

- Tool orchestration — Sequencing several tools to complete one goal, and handling partial failure sensibly.
- Hierarchical sub-agents — A parent delegates a bounded task to a child agent with its own context, tools and instructions.
- Context isolation — Sub-agents keep the parent's context clean by returning only the conclusion, not the whole working trace.

Lab 10 — Build and Connect an MCP Server

Learning outcome: Orchestrate tools through the Model Context Protocol.

Goal: Write your own MCP server exposing typed tools, then connect it to a coding agent. Because MCP is an open standard, the same server works with any MCP-aware client — write the capability once, use it everywhere.

What you'll build

A working MCP server with three typed tools, connected to and callable from your coding agent. (Tools: Python 3.11+, FastMCP (mcp package), Claude Code or another MCP client.)

Where the code lives

- Lab folder: labs/lab-10-build-and-connect-an-mcp-server/ — the runnable Python script, a README and the notebook.
- Run it locally: activate your virtual environment, set your keys in .env, then run the script named in the lab's README.
- Or open it in Google Colab: https://colab.research.google.com/github/tertiarycourses/TGS-2020503207-AI-Vibe-Coding-for-Multi-Agents-System/blob/main/labs/lab-10-build-and-connect-an-mcp-server/lab-10-build-and-connect-an-mcp-server.ipynb

Step-by-step

1. Install the MCP Python SDK and create server.py.

```
uv pip install "mcp[cli]"
```

2. Instantiate FastMCP and define your first tool with the @mcp.tool() decorator, using type hints and a clear docstring.
3. Add two more tools — one reading local data and one calling an external API.
4. Run the server over stdio and confirm it starts without error.

```
python server.py
```

5. Register the server with your MCP client so the agent can discover it.

```
claude mcp add my-tools -- python /absolute/path/to/server.py
```

6. List the available tools from the client to confirm discovery.

```
claude mcp list
```

7. Ask the agent a question that requires your tools, and watch it select and call them.
8. Return a clear error message from a tool on bad input and confirm the agent handles it gracefully.

Test it

The MCP client lists all three tools, and the agent completes a task that is impossible without them, calling the tools in a sensible order.

Note: The complete working code for this lab is in labs/lab-10-build-and-connect-an-mcp-server/ in the course repository. Keep your API keys in .env — never commit them.

Lab 11 — Hierarchical Sub-Agents and Context Isolation

Learning outcome: Design a hierarchical sub-agent architecture with delegated, isolated context.

Goal: Build the top of the architecture: a parent agent that delegates bounded tasks to specialised child agents, each with its own context and tools. The children return conclusions, not their whole working trace, which is what keeps a large system reliable.

What you'll build

A three-level hierarchical system — orchestrator, sub-agents and MCP tools — that completes a multi-part task no single agent handles well. (Tools: Python, openai-agents SDK or google-adk, your MCP server from Lab 10.)

Where the code lives

- Lab folder: labs/lab-11-hierarchical-sub-agents-and-context-isolation/ — the runnable Python script, a README and the notebook.
- Run it locally: activate your virtual environment, set your keys in .env, then run the script named in the lab's README.
- Or open it in Google Colab: https://colab.research.google.com/github/tertiarycourses/TGS-2020503207-AI-Vibe-Coding-for-Multi-Agents-System/blob/main/labs/lab-11-hierarchical-sub-agents-and-context-isolation/lab-11-hierarchical-sub-agents-and-context-isolation.ipynb

Step-by-step

1. Define the parent orchestrator whose only job is to decompose the task and delegate.
2. Expose each specialist sub-agent to the parent as a callable tool.
3. Give each sub-agent its own instructions and only the tools it needs — including your MCP tools from Lab 10.

4. Ensure each sub-agent returns a short structured conclusion rather than its full transcript.
5. Run sub-agents concurrently where their work is genuinely independent, and measure the time saved.
6. Give the orchestrator a task requiring at least three sub-agents and trace the delegation.
7. Compare the parent's final context size with and without isolation to see the benefit quantitatively.
8. Handle a failing sub-agent so one failure degrades the result instead of killing the run.

Test it

The orchestrator completes the multi-part task by delegating to at least three sub-agents; the parent's context stays small because children return conclusions only, and one deliberately failing sub-agent does not crash the run.

Note: The complete working code for this lab is in labs/lab-11-hierarchical-sub-agents-and-context-isolation/ in the course repository. Keep your API keys in .env — never commit them.

Course Revision

Use this checklist to revise before the assessment. Each point maps to a learning outcome and is covered by both the slides and the labs.

- Describe the components of a modern AI agent and explain the reason-act-observe loop.
- Explain what agent skills and agent memory are, and when each is used.
- Explain what MCP is, what an MCP server exposes, and why the standard matters.
- State when a single agent is sufficient and when to split into a multi-agent system.
- List the stages of the structured vibe coding workflow and explain context engineering.
- Build an OpenAI Agents SDK agent with a function tool and a Pydantic output type.
- Build a triage agent that routes to specialist agents via handoffs.
- Build the equivalent coordinator and sub-agents in the Google Gemini ADK.
- Deploy a multi-agent system as a Streamlit application with session state.
- Write an MCP server with typed tools and register it with an MCP client.
- Explain hierarchical sub-agents and why context isolation improves reliability.

Assessment Preparation

- Written Assessment (WA) — Short-Answer Questions (SAQ), 50 minutes, open book.

- Practical Performance (PP) — hands-on multi-agent build tasks, 75 minutes, open book.
- The assessment is OPEN BOOK — bring this Learner Guide and the course slides.
- Complete every lab; the practical tasks are based directly on what you built in class.
- A minimum of 75% attendance is required to be eligible for assessment and funding.
- Submit your answers on the LMS at https://lms-tms.tertiaryinfotech.com, and complete the TRAQOM survey.

Glossary

- **Agent —** An LLM that reasons in a loop, calls tools and pursues a goal, rather than answering a single prompt.
- **Agent skill —** A modular, transferable capability — a documented procedure an agent loads when a task needs it.
- **Agent memory —** What the agent carries forward: short-term conversation context and long-term stored facts.
- **Tool / function calling —** A function the agent may invoke; the runtime executes it and returns the result into the loop.
- **MCP (Model Context Protocol) —** An open standard letting any compatible agent discover and call tools exposed by a server.
- **MCP server —** A program that publishes typed tools, resources and prompts over the Model Context Protocol.
- **Multi-agent system —** Several specialised agents collaborating, usually coordinated by a supervisor.
- **Supervisor / triage agent —** The agent that classifies an incoming request and routes it to the right specialist.
- **Handoff —** Delegation of a request from one agent to another in the OpenAI Agents SDK.
- **Sub-agent —** A child agent given a bounded task, its own tools and its own isolated context.
- **Context isolation —** Keeping a sub-agent's working detail out of the parent's context by returning only conclusions.
- **Structured output —** A validated, typed result (e.g. a Pydantic model) instead of free-form text.
- **Guardrail —** An input or output check that blocks unsafe or out-of-scope work before it proceeds.
- **Vibe coding —** Directing an AI coding agent in natural language to build software while you specify and review.
- **Context engineering —** Deliberately curating what the coding agent sees — the main driver of output reliability.
- **Runner —** The component that executes the agent loop (Runner.run_sync in the OpenAI SDK; Runner in ADK).

- **Streamlit —** A Python library for building simple web UIs, used here to deploy the agent systems.